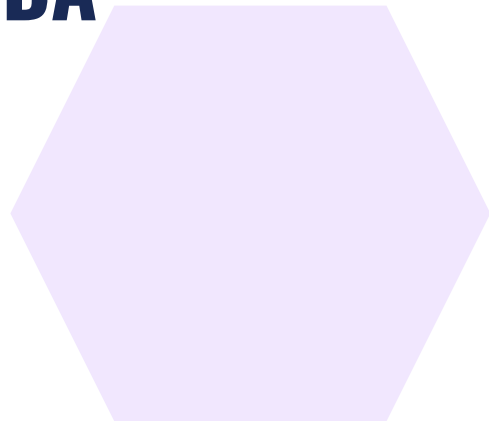




TLC TALK

ARQUITETURA DE SOFTWARE NA ERA DA IA

Complexidade local, complexidade global, e o que mudou





O CUSTO DA ABSTRAÇÃO RITUALÍSTICA É MENSURÁVEL

76%

acurácia do agente quando a dependência está escondida em DI/DIP

58%

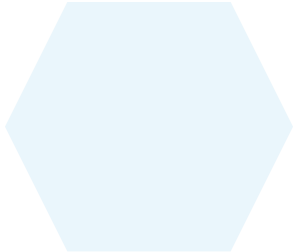
das vezes o agente ignora a ferramenta de navegação (heurística: glob+read já me dá 80%)

7-13

arquivos abertos por feature em estrutura multi-pasta (domain/app/infra)

ABSTRACTION BLOAT O LLM foi treinado com blogs sobre Event Sourcing, Hexagonal, Saga. Ninguém escreve sobre o CRUD de 3 arquivos que roda há 5 anos. **O viés é estrutural.**

Navigation Paradox (2026) · Addy Osmani: Abstraction Bloat · Super Productivity: Abstraction Illusion



PARTE 02

O QUE SOBROU

DDD estratégico ficou crítico. DDD tático precisa ser questionado peça por peça.



ESTRATÉGICO → FLAT → ABSTRAI POR DOR

01

ESTRATÉGICO PRIMEIRO

Pastas por domínio (billing, orders, identity). Cada um com seu modelo, sem importar do outro. DDD estratégico no nível mais barato: pasta. Sem framework, sem biblioteca.

02

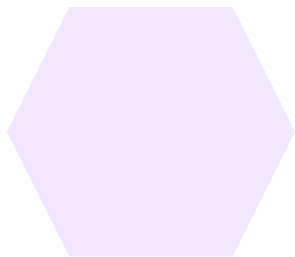
FLAT DEPOIS

Dentro do módulo, 3 responsabilidades claras: entrada/saída · lógica de negócio · persistência. Sem interface pra cada repo, sem use case pra cada CRUD, sem mapper entre 3 camadas de DTO.

03

ABSTRAI POR DOR

Trocou a dependência nos últimos 2 anos? Tem 2+ usos? Risco real de vazamento (gateway de pagamento)? Só então abstrai. Antes disso é YAGNI.



PARTE 03

ESCALA MUDA TUDO

Complexidade local vs complexidade global



MONÓLITO · MODULAR · MICROSERVIÇOS

MONÓLITO

Granularidade grossa

- + Simplicidade operacional
- + Transações ACID
- + Debug fácil
- Escala tudo ou nada
- Acoplamento acidental
- Complexidade LOCAL alta

MODULAR

Sweet spot · granularidade média

- + Visibilidade total (monorepo)
- + Deploy flexível por app
- + Evolução gradual e reversível
- Requer governança
- Ferramental (Nx/Bazel)
- Disciplina de fronteiras

MICROSSERVIÇOS

Granularidade fina

- + Autonomia máxima de times
- + Escala granular
- + Deploy isolado
- Complexidade GLOBAL alta
- APIs congelam
- Mudanças atômicas impossíveis

ERRO FUNDAMENTAL Microserviços conflitam fronteira lógica e física. Modular separa as duas.



MICROSSERVIÇOS NA ERA DA IA: 7 DESAFIOS

01

CONTEXT FRAGMENTATION

A IA enxerga 1 serviço de 30. O gargalo pode estar onde ela nunca olhou.

02

REFACTORS CROSS-SERVICE

Renomear campo de evento toca N serviços. Sem garantia de cobertura nem ordem.

03

CONVENTION DRIFT

Logging, retry, error handling: a IA reinventa o padrão a cada repo.

04

INTEGRATION TESTS

Unit é trivial. Mesh com traces distribuídos e mocks de N serviços não é.

05

DEV LOCAL

docker-compose com 15 serviços já era ruim. Agora a IA precisa rodar pra validar.

06

OBSERVABILIDADE

A resposta vive em traces distribuídos. Maioria dos times não conectou esse acesso à IA.

07

REVIEW EM ESCALA

Agente abre PR em 10 serviços. Quem revisa? Quem mantém a coerência arquitetural?

CADA UM DESTES

amplifica a complexidade global e cobra esse custo a cada feature.



4 FASES, DECISÕES REVERSÍVEIS

01

MONÓLITO MODULAR

1 app, todos os módulos juntos. Fronteiras lógicas via pastas. Velocidade máxima.

02

FRONTEIRAS CLARAS

Mesma app, governança forte: `enforce-module-boundaries`, observabilidade por módulo, feature flags.

03

APPS DEDICADAS

Módulos críticos promovidos para apps próprias. Deploy e escala independentes. Apps menores convivem.

04

MICROSSERVIÇOS

Apenas quando compliance ou escala extrema justificar. Comunicação só via rede. Custo operacional alto.



SEM ESTRUTURA, IA GERA CAOS 10X MAIS RÁPIDO

O ARGUMENTO DO OUTRO LADO

Se IA gera código 10x mais rápido, ausência de estrutura vira caos 10x mais rápido.

Clean Architecture não é só sobre o dev navegar. É sobre o sistema se manter coerente quando agentes produzem código o dia inteiro.

Sem limites explícitos, a IA vai gerar inconsistência em escala.

LIMITES VÊM DE LUGARES BARATOS

Concordo que limites importam. Discordo que custe 18 arquivos por feature.

- ✓ Bounded contexts bem definidos
- ✓ Pasta por domínio (DDD estratégico barato)
- ✓ Tipos fortes no contrato entre módulos
- ✓ Lint rules impedindo import cross-domínio
- ✓ Testes de comportamento (não implementação)
- ✓ Linguagem ubíqua nos prompts

A PROVOCAÇÃO FINAL

FUNDAMENTO OU HÁBITO?

Você consegue justificar cada camada de abstração com o problema real que ela resolve?